Concurrencia y Paralelismo

Grado en Informática 2026

Práctica 3 – Paso de mensajes

Ejercicio 1 (**Queue**) Implemente un módulo `queue` que implemente una cola en Erlang. No es necesario crear un proceso para guardar la cola.

La cola debe implementarse basándose en dos listas que mantengan, respectivamente, el principio y el final de la cola. Por ejemplo, considere la cola `[1,2,3,4,5]`, donde 1 es el elemento al frente de la cola, y 5 el último. Una posible representación con dos listas sería `[1,2,3]` y `[5,4]`, donde el principio de la cola está en orden, y el final al revés.

Insertar un elemento es una operación O(1), porque solo hay que añadirlo al principio de la segunda lista. Por ejemplo, añadir 6 daría `[1,2,3]` y `[6,5,4]`. Eliminar un elemento es más complejo, porque hay 2 situaciones:

- La primera eliminación quitaría el primer elemento de la primera lista (O(1)), y quedarían como `[2,3]` y `[6,5,4]`. Las dos siguientes eliminaciones también serían O(1), y las dos listas quedarían como `[]` y `[6,5,4]`
- La siguiente eliminación no puede usar la primera lista porque está vacía. El siguiente elemento está al final de la segunda. En esta situación se le da la vuelta a la segunda lista, y se cambia por la primera. Es decir, se pasa de `[]` y `[6,5,4]` a `[4,5,6]` y `[]`. Ahora se puede hacer la eliminación nuevamente quitando el primer elemento de la primera lista. Esta operación tiene complejidad O(n), porque hay que darle la vuelta a la segunda lista. Pero garantiza que las siguientes n eliminaciones son O(1), por lo que el coste medio es O(1) para todas ellas. Este tipo de complejidad algorítmica se denomina complejidad amortizada.

El API debe ser el siguiente:

- `queue:empty()`, devuelve una cola vacía.
- `queue:insert(Queue, Elem)`, donde Queue es una cola, y Elem un elemento, inserta el elemento en la cola, y devuelve la cola resultante.
- `queue:remove(Queue)` elimina el elemento al frente de la cola. Debe devolver una tupla `{ok, Elem, Queue}`. Si la cola está vacía, devuelve el átomo empty.

Por ejemplo:

```
1> S0 = queue:empty().
{[], []}
2> queue:remove(S0).
empty
3> S1 = queue:insert(S0, a).
{[], [a]}
4> S2 = queue:insert(S1, b).
{[], [b,a]}
5> {ok, Elem, S3} = queue:remove(S2).
{ok, a, {[b], []}}
6> queue:remove(S3).
{ok, b, {[], []}}
```

Ejercicio 2 (**Procesos en línea**) Implemente un módulo `line`, que cree una secuencia de procesos, donde cada proceso conoce el PID del siquiente en la secuencia. El módulo debe proporcionar las siguientes funciones:

- `line:start(N)`, donde N es el número de procesos de la secuencia, y devuelve el PID del primer proceso.
- `line:send(Pid, Msg)`, donde Pid es el Pid del primer proceso en una secuencia y Msg es un término Erlang cualquiera. Cada proceso debe imprimir su número en la secuencia, el mensaje recibido, y pasar el mensaje al siguiente proceso. Cuando el mensaje llegue al último se imprime pero se para la propagación.
- `line:bounce(Pid, Msg, Times)`, donde el mensaje se transmite hacia el final de la linea decrementando Times cada vez que se avanza un proceso. Si al llegar al final Times es mayor que 0, el mensaje debe volver hacia atrás. Si vuelve a llegar al principio, debe volver hacia adelante.
- `line:stop(Pid)`, que para todos los procesos de la secuencia.

Por ejemplo:

```
1> P = line:start(3).
<0.84.0>
2> line:send(P, hola).
0 received message hola
ok
1 received message hola
2 received message hola
3> line:bounce(P, hola, 6).
0 received message hola
ok
1 received message hola
2 received message hola
1 received message hola
0 received message hola
1 received message hola
```

El átomo ok que devuelve como resultado line:send y line:bounce puede aparecer en cualquier posición dentro de la secuencia de mensajes impresos.

Ejercicio 3 (**Anillo de procesos**) Implemente un módulo `ring`, que cree un anillo de procesos, donde cada proceso conoce el PID del siquiente en el anillo. El módulo debe proporcionar las siguientes funciones:

- `ring:start(N)`, donde N es el número de procesos en el anillo, y devuelve el PID del primer proceso.
- `ring:send(Pid, N, Msg)`, donde Pid es el Pid del primer proceso en un anillo, N el número de nodos que procesarán el mensaje, y Msg un término Erlang cualquiera. Cada proceso debe imprimir su número en el anillo, el número de mensajes por imprimir y el mensaje recibido. Si el número de mensajes por imprimir es mayor que 0 también enviará el mensaje al siguiente proceso. El mensaje puede dar más de una vuelta al anillo si N es mayor que el número de procesos en el anillo.
- `ring:stop(Pid)`, donde Pid es el primer proceso de un anillo, para todos los procesos de un anillo.

Por ejemplo:

```
1> R = ring:start(3).
<0.84.0>
2> ring:send(R, 5, hola).
0 receiving message hola with 4 left
ok
1 receiving message hola with 3 left
2 receiving message hola with 2 left
0 receiving message hola with 1 left
1 receiving message hola with 0 left
```

El átomo ok que devuelve como resultado ring:send puede aparecer en cualquier posición dentro de la secuencia de mensajes impresos.

Entrega

La fecha límite de entrega es el 20 de marzo. Para crear el respositorio y acceder al código inicial, acceda a github classroom en https://classroom.github.com/a/Zj67NFRQ. El nombre de los grupos de prácticas debe ser el login de ambas personas, separados por un guión.